CYBERSECURITY TRACK • PHP & WEB MODULE

Assignment 10

HTTP Request Handler & REST API Builder (PHP)

Done by: Abdulhadi Al-Ahdal

1. Project Overview

The objective of this assignment is to develop a REST API in PHP designed to handle different types of HTTP requests such as GET, POST, PUT, and DELETE[cite: 2]. The tool examines incoming headers, reads inputted JSON data, and returns customized responses with appropriate status codes to solidify concepts of request analysis and client-server communication[cite: 2].

2. Development Steps

1 JSON Content-Type Header

Programmatically set the returned response header to JSON format using `header("Content-Type: application/json")` [cite: 2].

2 Request Method & Body Extraction

Read incoming request methods using `$_SERVER['REQUEST_METHOD']` and extract body data with `file_get_contents("php://input")` and `json_decode` [cite: 2].

3 Conditional Routing

Build a conditional structure using `switch` or `if/else` to differentiate between allowed request methods[cite: 2].

4 HTTP Status & Response Output

Return appropriate HTTP status codes (e.g., 200 on success, 405 for unsupported methods) and print structured JSON output[cite: 2].

3. Technical Features & Utilities

Header Inspection: Use `getallheaders()` to read and track security headers such as `Authorization` [cite: 2].

Formatted JSON Encoding: Use `json_encode($response, JSON_PRETTY_PRINT)` to output responses in a clean, readable layout[cite: 2].

4. Implementation Source Code

`api.php`

PHP 8.x

```php
<?php
// Set response header to application/json
header("Content-Type: application/json; charset=UTF-8");

// Retrieve all headers including security tokens like Authorization
$headers = function_exists('getallheaders') ? getallheaders() : [];
$authHeader = $headers['Authorization'] ?? $headers['authorization'] ?? 'None';

// Retrieve request method
```

```php
$method = $_SERVER['REQUEST_METHOD'];

// Extract JSON body input safely
$rawInput = file_get_contents("php://input");
$data = json_decode($rawInput, true);

$response = [
    "status" => "success",
    "method" => $method,
    "auth_received" => ($authHeader !== 'None') ? "Valid Header Present" : "Missing Authorization",
    "timestamp" => date("Y-m-d H:i:s"),
    "data" => $data ?? []
];

// Switch statement to handle different HTTP methods
switch ($method) {
    case 'GET':
        $response["message"] = "GET request processed successfully. Resource data retrieved.";
        http_response_code(200);
        break;

    case 'POST':
        $response["message"] = "POST request processed successfully. New resource created.";
        http_response_code(201);
        break;

    case 'PUT':
        $response["message"] = "PUT request processed successfully. Resource updated.";
        http_response_code(200);
        break;

    case 'DELETE':
```

```
        $response["message"] = "DELETE request processed successfully. Resource removed.";
        http_response_code(200);
        break;

    default:
        http_response_code(405); // Method Not Allowed
        $response["status"] = "error";
        $response["message"] = "Method not allowed. Supported methods: GET, POST, PUT, DELETE.";
        break;
}

// Output formatted response JSON
echo json_encode($response, JSON_PRETTY_PRINT);
?>
```

5. Acceptance Criteria

- ✓ **Accurate Method Differentiation**
  Success in distinguishing between HTTP methods (GET, POST, PUT, DELETE) and responding accurately[cite: 2].

- ✓ **Robust Payload Parsing**
  Extraction and reading of request headers and JSON payloads without parsing errors, handling exceptions correctly[cite: 2].